



EarthLens AI – AI-powered Satellite Geo-Analytics for Urban Heat and Air Quality

Project Title and Executive Summary

Project Title: *EarthLens AI* – Interactive Satellite and Sensor Data Platform for Urban Heat Islands and Air Quality Monitoring. EarthLens AI is a one-person project that delivers an integrated Earth Observation system for Indian cities. It will combine open satellite imagery (Sentinel-2, Landsat, Sentinel-5P) and ground data (IMD weather, OpenAQ sensors) to monitor vegetation, surface temperature, and pollutants (e.g. PM2.5, formaldehyde). The platform offers an interactive map dashboard (built with React and Leaflet), REST APIs (Python/FastAPI), and geospatial storage (PostgreSQL/PostGIS). Core features include NDVI/LST mapping for urban heat islands, AQI and HCHO hotspot detection, basic forecasting (temperature and pollution trends), and later an AI assistant for data queries. An MVP (6–9 months) will focus on data ingestion, mapping, and simple forecasts; advanced phases will add anomaly alerts and recommendations. EarthLens AI is designed as a serious capstone/portfolio project demonstrating full-stack, ML, and GIS integration. It is **solely built by one developer**, with a phased plan to balance scope and resource constraints.

Problem Statement

Rapid urbanization and climate change are driving up **urban heat island (UHI)** intensity and air pollution in Indian cities, posing serious health and infrastructure risks. Urban residents face higher daytime temperatures (often several degrees above rural areas) and stagnant air pollution. According to WHO, heatwaves claimed over *166,000 lives* worldwide from 1998–2017, with India suffering a rising toll in recent decades [36†L147-L152]. Meanwhile, pollutants like fine particulates and volatile organic compounds (e.g. formaldehyde) exacerbate respiratory and cardiovascular diseases. City planners lack integrated data tools to pinpoint where heat stress and pollution coincide, forecast their evolution, or guide mitigation (like greening or cooling). Most current dashboards focus on either weather or pollution alone and do not leverage high-resolution satellite observations, machine learning forecasts, or actionable recommendations tailored to urban settings.

Why This Project Is Needed

EarthLens AI addresses a critical need for data-driven urban resilience: it provides **high-resolution spatial monitoring** of thermal and air quality hazards, combined with short-term predictions and planning support. Existing tools (government AQI websites, static heat maps) are often siloed, coarse, or reactive. In contrast, EarthLens AI will fuse satellite NDVI/LST data (indicating vegetation cover and surface temperature) with ground AQI measurements to reveal **where and when** cities are hottest and most polluted. By automating data pipelines and AI models, it empowers stakeholders with evidence-based insights—e.g., identifying neighborhoods with sparse trees and high heat (low NDVI correlated with high surface temperature [14†L121-L124]) or industrial zones emitting VOC hotspots. This integrated platform

can inform Heat Action Plans, urban greening, and public alerts. Its AI assistant (a chatbot layer) will make the complex data accessible to non-experts. Overall, EarthLens AI fills a gap: an **open, end-to-end environmental intelligence system** for cities, leveraging the latest research (deep learning forecasting, geospatial analytics) in a practical, deployable tool.

Current Solutions and Their Limitations

Existing solutions tend to focus narrowly on one aspect: for example, India's **SAFAR** and CPCB AQI dashboards show pollution levels but lack detailed spatial analysis or future projections. GIS tools (like Esri's Urban Heat mapping) can produce UHI vulnerability maps [14†L121-L124] , but they are commercial and not easily accessible for open use or non-specialists. Global platforms like OpenAQ provide crowdsourced air quality data [6†L112-L120] , yet coverage in many Indian cities is sparse. Satellite products (e.g. NASA Earth Observatory, NOAA climate data) exist but require expert interpretation. Crucially, none of these integrates urban heat (NDVI/LST) with air pollution (HCHO, PM2.5) in a unified, interactive interface. They also typically lack predictive analytics; forecasts rely on coarse weather models that don't capture intra-city variability [36†L147-L152] . EarthLens AI bridges these gaps by combining all relevant data sources into a **single system**, adding machine-learning forecasts and recommendations. In short, current dashboards and apps handle slices of the problem, whereas EarthLens AI offers an integrated, intelligent approach.

Project Vision and Objectives

Vision: To equip Indian city decision-makers and communities with a smart, data-driven lens on their environment. EarthLens AI will function as an “**urban climate digital twin**”, visualizing real-time and historical heat and pollution patterns and projecting short-term trends.

Objectives:

- **Map and Monitor UHI:** Calculate NDVI (vegetation index) and LST (land surface temperature) from Sentinel-2/Landsat to visualize heat islands at neighborhood scale [25†L61-L65] [23†L68-L72] .
 - **Analyze Air Quality Hotspots:** Ingest ground-based AQI data (PM2.5, PM10, etc.) and atmospheric HCHO from Sentinel-5P [17†L68-L74] [21†L112-L118] to locate pollution hotspots (e.g. industrial clusters, traffic corridors).
 - **Forecast Environmental Trends:** Develop ML models (e.g. LSTM networks) to predict short-term temperature and AQI patterns. (LSTM has shown high accuracy for urban temperature forecasting [31†L68-L72] .)
 - **Interactive Web Dashboard:** Build a user-friendly React app with an interactive map (showing layers for heat, AQI, HCHO) and charts, enabling users to query and visualize data over time.
 - **Actionable Insights:** Provide alerts (e.g. extreme heat advisories), recommendations (e.g. where to plant trees), and an AI chatbot to answer questions about observed patterns.
 - **Low-Cost, Open-Source Platform:** Use free or open datasets and tools (Google Earth Engine, OpenAQ API, OpenWeatherMap) to keep development costs near zero.
- Each objective is phased to suit a single developer: initial focus on data gathering and basic mapping, with later phases adding forecasting, optimization, and the AI layer.

Who the Product Is For

- **Urban Planners and Municipalities:** To identify and prioritize heat-vulnerable wards and pollution sources, guiding interventions (cool roofs, green spaces, traffic control).
- **Environmental Researchers and NGOs:** To access integrated datasets and trend analysis without building infrastructure themselves.
- **Public Health Officials:** To correlate heat and air quality data with health outcomes and to plan early warnings (e.g. heatwave alerts).
- **Citizens and Educators:** To explore local environmental conditions via a friendly map interface and chatbot, raising awareness and community engagement.
- **Tech/Startup Practitioners:** As a prototype that could seed a startup or a comprehensive final-year project, demonstrating how to build full-stack geospatial AI tools.

What the Product Will Do

EarthLens AI will:

- **Ingest Data:** Automatically fetch satellite images (Sentinel-2, Landsat, Sentinel-5P) and ground/ weather data (IMD, OpenWeatherMap, OpenAQ) on a regular schedule.
- **Process and Store:** Compute NDVI and LST from images, clean sensor readings, and store time-series in a PostGIS database.
- **Visualize:** Render interactive maps where users select a city/date and view layers like heatmaps of LST, vegetation index (NDVI), AQI levels, and HCHO concentration. Pop-ups or side panels will show statistics for clicked locations.
- **Forecast:** Generate short-term (e.g. 7–14 day) forecasts of city-scale peak temperature and AQI using ML models (see *ML/DL Models* below). Display these as future trend charts and “expected heat index”.
- **Detect Hotspots:** Identify spatial hotspots where values exceed thresholds (e.g. 95th percentile) or change abruptly, flagging them on the map.
- **Recommend:** Based on combined heat and pollution layers, suggest mitigation (e.g. areas for urban greening) and issue alerts if predictions exceed safety thresholds.
- **Answer Questions:** An AI chatbot (phase 3) that answers user queries like “Why is Sector 5 so hot?” or “Where are AQI violations predicted next week?” by querying the backend and/or using a language model.

Overall, the product turns complex environmental data into actionable, location-specific insights via an intuitive dashboard.

Scope of the Project for a Solo Developer

One-person Reality: Given this is a solo project, the scope must be carefully controlled. All features should be achievable by one developer using open resources. Key adjustments include:

- **Focus on a Few Cities:** Initially target 1–3 cities (e.g. Nagpur, Pune, Mumbai) rather than trying to cover all of India at once. This limits data volume and simplifies validation.
- **Phased Rollout:** Prioritize core functionality (data ingestion, mapping) for the MVP. Defer advanced ML modeling and AI until later phases.

- **Reuse and Libraries:** Leverage existing platforms (Google Earth Engine for satellite processing, OpenAQ API for air data) instead of building custom processing from scratch. Use open-source libraries (Python GDAL, scikit-learn/TensorFlow, React components).
- **Minimalist Infrastructure:** Use a simple cloud or even a laptop+low-cost server, avoiding complex distributed systems. For example, process most satellite data via Google Earth Engine (no server cost) and use a single PostgreSQL/PostGIS instance (on a free-tier cloud VM).
- **Reasonable Timeline:** Plan on 6–12 months of part-time work (or ~3–6 months full-time) for an MVP, followed by additional months for enhancements. Strictly define what constitutes “done” in each phase to avoid feature creep.

By narrowing the focus (e.g. one pilot city), relying on managed services (GEE, external APIs), and phasing development, EarthLens AI can be built by one person without sacrificing the core vision.

Features Planned for the MVP

The **MVP (Minimum Viable Product)** will include the essential features to demonstrate EarthLens AI's value:

- **Data Pipeline Setup:** Scripts to fetch and preprocess satellite imagery for one city. For example, use Google Earth Engine to compute recent NDVI and LST composites [25+L61-L65] [23+L68-L72] . Pull current AQI data (PM2.5, O3, etc.) from OpenAQ and weather from OpenWeatherMap. Store cleaned data in PostGIS tables.
- **Basic Web Dashboard:** A React-based UI with a map (using Leaflet or Mapbox GL) displaying: (a) an NDVI layer (showing vegetation), (b) an LST heatmap (urban heat), and (c) an AQI overlay (e.g. PM2.5). Controls allow the user to toggle layers and select a date range. Clicking on the map shows values for that point.
- **City Comparison Charts:** Simple charts (Chart.js or D3) to display time series of average temperature or AQI in the city over the past month.
- **Lookup Tools:** A search box or dropdown to select the city and date, updating the map.
- **Initial Predictions:** A placeholder prediction feature, e.g. a linear regression model forecasting next-week temperature/AQI based on recent trends, with results shown in a chart. (Even a naive model is fine for MVP.)
- **Basic API Endpoints:** A small set of FastAPI endpoints to serve map data (e.g. `/api/heatmap?city=Nagpur&date=2026-07-01`) and chart data (`/api/timeseries?city=Nagpur&type=heat`).
- **Documentation:** Commented code and a README so others can run the system.

These MVP features provide a working proof-of-concept: an interactive map showing heat and air quality data for a city, with the backend gathering data and serving the dashboard. Everything else (AI assistant, sophisticated models, alerts) can wait.

Future Features That Can Be Added Later

Beyond the MVP, phased future enhancements could include:

- **Advanced Forecasting:** Replace naive models with robust ML. For heat, use LSTM or GRU networks to forecast daily maximum LST or heat index. For air quality, consider time-series models (Prophet or

LSTM) for PM2.5. (Research shows LSTMs can achieve $R^2 \approx 0.98$ on urban temperature data [31†L68-L72]). Implement retraining pipelines for these models.

- **Hotspot Detection & Alerts:** Algorithms to detect spatial anomalies (e.g. high-HCHO pockets) using clustering or statistical thresholds, with automated alerts sent (email/SMS) if critical thresholds are exceeded.
- **AI Assistant (Chatbot):** Integrate a chatbot (e.g. a fine-tuned GPT) that interfaces with the API. It can answer natural-language queries about the data (drawing from maps, charts, and predictive models). For example, "Show me the hottest neighborhoods next week," or "Is pollution worse near factories or roads?"
- **Additional Data Layers:** Incorporate more satellite indices or data sets, such as Normalized Difference Built-up Index (NDBI) for impervious surface, or precipitation forecasts. Integrate crowdsourced weather or pollution sensor networks if available.
- **User Accounts and Customization:** Allow users to save settings (favorite cities), annotate the map, or subscribe to alerts for specific locations.
- **Mobile-Friendly UI:** Make the frontend responsive or create a mobile app wrapper for field use.
- **Scalability:** Extend coverage to many cities by automating data collection jobs and optimizing the database for multiple regions.
- **Visualization Enhancements:** Add layers like population density or socio-economic data to correlate with heat vulnerability. Incorporate 3D maps or time-lapse animation for impact.
- **Integration with City Planning Tools:** Export data to GIS formats or link with urban planning frameworks (e.g. data feeds for the Smart Cities Dashboard).

These features can be tackled incrementally once the MVP is stable, allowing EarthLens AI to grow without overwhelming a lone developer.

Detailed System Architecture

The system follows a standard full-stack architecture: **data sources** → **data pipeline** → **database** → **API layer** → **frontend**. Below is a high-level view:

- **Data Sources (External):**
 - *Google Earth Engine (GEE):* Provides satellite imagery (Sentinel-2, Landsat, Sentinel-5P) and built-in computation.
 - *OpenAQ and Other APIs:* Global air quality (OpenAQ) [6†L112-L120] , weather (OpenWeatherMap), official IMD forecasts if accessible.
 - *Static GIS Data:* City boundaries, roads, etc. (from official shapefiles or OpenStreetMap).
- **Backend Services:**
 - *Data Ingestion Scripts:* Python scripts (possibly scheduled or on-demand) using GEE Python API to fetch NDVI/LST/HCHO, and requests to OpenAQ/OpenWeather APIs. These scripts perform preprocessing and load data into the database.
 - *FastAPI Server:* Exposes RESTful endpoints (JSON) for frontend and AI assistant. Handles authentication (if any), queries the database or triggers on-the-fly calculations (e.g. generating a heatmap tile). Implements caching for expensive queries.

- **Machine Learning Modules:** Encapsulated model training and inference code. Models (e.g. LSTM weights) are stored and loaded by the FastAPI server to provide forecasts on request.
- **Database (PostgreSQL + PostGIS):**
All preprocessed data is stored here. Key tables include raw sensor readings (with timestamps), computed indices (NDVI, LST values by location/date), and city geometries. PostGIS spatial indexing is used for queries like “points within city boundary” or “distance to hotspot” [46†L17-L21] .
- **Frontend (React App):**
A single-page application using React. It fetches data from the backend APIs and displays maps, charts, and UI controls. Mapping libraries (e.g. Leaflet or Mapbox) render layers (heatmaps, markers). Chart libraries plot time series. The UI communicates user actions (e.g. selecting a date or toggling layers) back to the API.
- **CI/CD & Deployment:**
The codebase can be containerized (Docker). Deployment may use free/low-cost platforms (e.g. deploying the API and DB on a free-tier cloud VM, hosting the frontend on a static site host). A simple continuous integration pipeline (GitHub Actions) can run tests and deploy to staging/production on pushes.

This modular architecture allows swapping components: for example, one could replace GEE with local processing, or swap React for another frontend if needed. It also ensures that heavy lifting (satellite processing) is offloaded (to GEE), while the FastAPI handles user-facing logic.

Data Sources and Why Each One Is Used

EarthLens AI combines multiple public data sources, each chosen for relevance and cost-effectiveness:

- **Sentinel-2 (ESA):** A free multispectral satellite with 10-meter visible/NIR bands [25†L61-L65] . Its red (Band 4) and NIR (Band 8) bands enable NDVI calculation to measure vegetation. High revisit (~5 days) provides fresh imagery, useful for tracking green cover and its cooling effect on UHI. [25†L61-L65] NDVI is crucial for identifying tree-poor hotspots and validating green interventions.
- **Landsat 8/9 (USGS/NASA):** Public-domain satellites with thermal infrared bands. Earth Engine’s Landsat Collection 2 Level-2 includes atmospherically-corrected *Land Surface Temperature* (LST) data [23†L68-L72] . This gives direct pixel-wise surface temperature (in Kelvin) to map heat islands. We use Landsat’s thermal band because Sentinel-2 lacks thermal channels. LST is the primary metric for UHI intensity.
- **Sentinel-5P (Copernicus):** A European atmospheric monitoring satellite carrying the TROPOMI sensor. It provides daily global maps of trace gases. In particular, the “Formaldehyde” (HCHO) product [21†L112-L118] shows tropospheric HCHO column densities. HCHO is an intermediate gas from vehicle and industrial VOC emissions [21†L112-L118] . We use Sentinel-5P to identify volatile organic hotspots (e.g. near factories or heavy traffic).

- **OpenAQ (Ground Sensors):** An open platform aggregating government and research air quality monitors globally [6†L112-L120] . It provides near-real-time measurements of PM2.5, PM10, NO₂, O₃, etc., for many Indian cities. EarthLens AI uses OpenAQ to fill gaps between satellite overpasses and to capture pollutants not seen from space. For example, combining OpenAQ PM2.5 with satellite HCHO gives a more complete pollution picture.
- **OpenWeatherMap / IMD (Meteorology):** Weather data (temperature, humidity, wind) is needed both for contextual insights and to inform models. OpenWeatherMap's free API provides global forecasts and historical weather for cities. The India Meteorological Department has official data (if accessible via APIs [29†L133-L141]), but may require sign-up. We will start with OpenWeatherMap (free tier up to 60 calls/min) for simplicity. Meteorological context helps interpret heat/pollution data (e.g. wind dispersing pollutants, humidity affecting heat index).
- **Local Datasets (optional):** If available, city shapefiles (wards/blocks) or population density maps could be included for vulnerability analysis. These would be static GIS files, serving as backdrops or for stratification.

All these datasets are **public or free to use**, aligning with the cost constraints. Notably, Google Earth Engine acts as a hub for downloading and computing on satellite data, sparing us from manual image handling and server storage. We cite Google's GEE catalog pages [23†L68-L72] [25†L61-L65] [21†L112-L118] to show how it provides preprocessed NDVI/LST/HCHO data readily.

APIs, Datasets, and Pretrained Models

APIs:

- *Google Earth Engine Python API:* To programmatically retrieve and compute NDVI/LST/HCHO layers. Earth Engine simplifies access to entire Sentinel and Landsat archives.
- *OpenAQ API* [6†L112-L120] : For air quality monitors (no authentication needed).
- *OpenWeatherMap API:* For current and forecasted weather.
- *IMD APIs* [29†L133-L141] (if licensing allows) for high-quality local forecasts and historical data.
- *Maps/Geocoding APIs* (optional): e.g. Mapbox or OpenStreetMap API for base maps and city boundary fetching (usually free up to a limit).

Datasets:

- *Earth Engine Public Datasets:* We will use the publicly available collections described above (Sentinel-2 L2A, Landsat 8 L2, Sentinel-5P OFFL HCHO) [23†L68-L72] [25†L61-L65] [17†L68-L74] . These are built-in to Earth Engine.
- *City Boundaries:* Shapefiles or GeoJSON from Indian government or OpenStreetMap for defining city extents.
- *Ground Sensor Archives:* Bulk OpenAQ data on AWS (S3) could be used for historical data if needed, as pointed out in OpenAQ docs.

Pretrained Models and ML Libraries:

- No pretrained neural networks are strictly required (satellite tasks are handled by Earth Engine). However, for the **AI assistant** we may use a pretrained language model (e.g. GPT-3/4 via OpenAI API, or an open alternative like Vicuna/LLaMA via Hugging Face). This would provide conversational capability without training from scratch.
- For **forecasting**, libraries such as Prophet (Facebook's time-series model) or scikit-learn (for Random Forests/Linear Regression) can serve as quick baselines. If deploying neural nets, TensorFlow or PyTorch would be used. (Citing [31†L68-L72] shows that LSTM outperforms Prophet in urban heat prediction, but we could start simpler.)

The choice of APIs and models is driven by the open/free requirement. All planned components have open-source or free-tier options, and where training is needed we rely on well-established libraries rather than proprietary tools.

Tech Stack with Justification

- **Python (Backend & Data Processing):** Python's rich ecosystem (Pandas, NumPy, scikit-learn, TensorFlow) makes it ideal for data analysis and ML. The [FastAPI](#) framework is chosen for the API server [42†L311-L319] due to its speed, ease of use, and built-in support for async and data validation. FastAPI's autogenerated docs (Swagger UI) help debugging and future expansion.
- **FastAPI:** As a Python framework, FastAPI is *modern and high-performance* [42†L311-L319]. It allows writing type-annotated endpoint functions, leading to faster development and fewer bugs. Its performance is on par with NodeJS/Go, which is beneficial for serving geospatial queries and ML inference.
- **React (Frontend):** React is a popular JavaScript library for building dynamic user interfaces [44†L16-L24]. Its component-based model fits well with an interactive map dashboard (map component, layer toggles, chart components). React also has a mature ecosystem (React-Leaflet, Mapbox GL React, Chart.js wrappers) that simplifies implementing the UI.
- **Leaflet or Mapbox GL (Mapping Library):** To render interactive maps in React, we will use Leaflet (with OpenStreetMap tiles) or the open-source MapLibre. These libraries easily display GeoJSON layers, heatmaps, and markers in a web map. They integrate with React via packages like `react-leaflet`.
- **PostgreSQL + PostGIS (Database):** PostgreSQL is a robust open-source relational database. The PostGIS extension adds geospatial types and indexes [46†L17-L21], crucial for this project. For example, PostGIS lets us store city boundaries or sensor points and perform spatial queries (e.g. "find all readings within this city polygon") efficiently [46†L17-L21]. Being open-source, PostgreSQL/PostGIS can be run on any server or local machine.
- **Google Earth Engine:** Not strictly a "tech stack component," but GEE is a core data source. Its Python API runs on the backend server (or even locally) to fetch processed satellite data. GEE's free tier is sufficient for research/portfolio use.

- **Machine Learning Libraries:**

- *scikit-learn* for classical ML (linear regression, random forests, clustering).
- *TensorFlow* or *PyTorch* for any deep learning (LSTM) used in forecasting. These have wide tutorials and community support.
- *Prophet* (by Meta) for time-series forecasting if simpler.
- *spaCy* or *transformers* for any NLP needed in the chatbot (if we build it ourselves).

- **DevOps / Deployment Tools:**

- *Docker* to containerize the application, ensuring reproducibility.
- *NGINX* (or *uvicorn/gunicorn*) as the WSGI server.
- *GitHub / Git* for version control. Possibly *GitHub Actions* for CI/CD.
- *Free hosting*: Tools like *Railway.app*, *Fly.io*, or free-tier AWS/GCP can host the backend+DB. The frontend can be served via *GitHub Pages* or *Netlify* for free.

Each technology is chosen for being open-source or free (FastAPI, React, PostgreSQL, etc.) and having strong community support. For instance, FastAPI's documentation claims it can “*spawn a REST server... for predictions*” at companies like Microsoft and Uber [42†L311-L319] , underscoring its suitability. React's tagline is “a JavaScript library for building user interfaces” [44†L16-L24] , exactly our need for an interactive dashboard. PostGIS is literally “*the geospatial extension for PostgreSQL*” [46†L17-L21] , making geographic queries straightforward.

Data Pipeline and Preprocessing Steps

1. **Satellite Data Retrieval:** Use the Google Earth Engine (GEE) Python API to query imagery for each target city and date range. For Sentinel-2, collect Level-2A images (surface reflectance). For Landsat 8/9, use the Collection 2 Tier 1 L2 surface temperature band. For Sentinel-5P, fetch L3 formaldehyde products [17†L68-L74] .
2. **Cloud Masking and Compositing:**
3. *Sentinel-2*: Apply the QA60 band (cloud mask) to filter out cloudy pixels. Then compute $NDVI = (B8 - B4) / (B8 + B4)$ for each cloud-free pixel.
4. *Landsat*: Use the SR Band 10 (thermal) provided by Earth Engine; it includes inherent atmospheric correction [23†L68-L72] . Convert to Celsius ($^{\circ}C = K - 273.15$). Use only scenes with <10% cloud cover or use the QA band to mask clouds.
5. *Sentinel-5P*: Use the OFFL (offline L3) HCHO column density bands. No cloud mask is needed beyond what the dataset already applied.
6. **Spatial Aggregation:** Depending on map zoom level, we may aggregate or tile data. For example, compute city-wide statistics (mean NDVI, max LST) by intersecting with city polygons via PostGIS (`ST_Contains`). Also prepare gridded heatmaps: divide the city area into e.g. 100m or 500m cells and average pixel values for smooth rendering.

7. **Temporal Alignment:** Ensure all data layers align in time. Satellite overpasses might not coincide with ground sensor readings; standardize to daily values. For each day (or monthly if needed), compute:
8. *NDVI Day*: median NDVI from all cloud-free pixels that day.
9. *Max LST*: the highest LST pixel (or mean/median) for that day.
10. *AQI Values*: the daily average (or hourly series) from OpenAQ.
11. *HCHO Column*: one or two daily values (from Sentinel-5P's 2-day global revisit).
12. **Join with Weather Data:** Retrieve daily max/min temperature and humidity from OpenWeatherMap or IMD for context or model input.
13. **Data Cleaning:** Handle missing data (e.g. interpolate or carry forward). Ensure all numerical data is in appropriate units ($\mu\text{g}/\text{m}^3$ for PM2.5, $^{\circ}\text{C}$ for temperature). Store timezone-consistent timestamps (India Standard Time).
14. **Database Loading:** Load processed data into PostGIS tables: point records for each sensor reading, raster values or aggregates for NDVI/LST by coordinate, etc. Use geometry columns for location-based queries (e.g. geo-index sensor data).
15. **Preprocessing for ML:** If building a model, assemble feature vectors. For example, for heat forecasting, use historical LST, NDVI, and weather as features with timestamps; split into train/test. Scale/normalize features as needed. Save any normalization parameters.

In practice, Earth Engine greatly simplifies steps 1–3: GEE provides ready-to-use NDVI and LST products [23†L68-L72] [25†L61-L65], so the pipeline mainly orchestrates data retrieval rather than image processing from scratch. The backend can call GEE's Python functions to get results or even schedule Earth Engine "exports" to Google Drive/Cloud Storage if large.

ML/DL Models and Why They Are Suitable

- **Time-Series Forecasting (LSTM, Prophet):** For predicting future temperature and pollution trends, we treat the data as time series. Recurrent neural networks like LSTM (Long Short-Term Memory) are well-suited to capture complex temporal patterns and seasonality. Research shows LSTM significantly outperforms simpler models in urban heat prediction [31†L68-L72] (e.g. achieving an R^2 of 0.98 on Delhi data). Therefore, an LSTM (in TensorFlow/PyTorch) will be implemented for heat forecasts if enough data is available. Prophet (Facebook's model) is a strong baseline for seasonal trends; initial tests may use it since it requires no training infrastructure. Ultimately, ensemble or hybrid approaches could be considered.
- **Regression Models:** As simpler baselines, linear regression or Random Forests can model relationships (e.g. LST vs. NDVI+weather). These require less data and computing power, so they can be used early on or if data is limited.

- **Hotspot Detection:** We may use unsupervised methods. For instance, clustering (DBSCAN) can find contiguous high-value regions of pollution or heat. Alternatively, statistical outlier detection (e.g. points above $\text{mean} + 2\sigma$) flags hotspots. These are computationally light and interpretable.
- **Classification (Alerts):** If labeling data (e.g. “heatwave day” vs. “normal day”), simple classifiers (logistic regression, decision trees) could predict alert conditions. However, for MVP this is optional.
- **AI Assistant (NLP Model):** A large language model (LLM) will power the chatbot. For cost, an API-based model (e.g. OpenAI GPT-4) could be used, prompting it with data from our API. Alternatively, an open LLM like LLaMA or GPT-J, possibly fine-tuned on domain Q&A, can be integrated using Hugging Face’s Transformers. This avoids the need to train an NLP model from scratch.

Each model choice balances solo-developer practicality with performance. We cite [31†L68-L72] to justify the use of LSTM for accuracy. In a one-person context, simpler models (Prophet, basic regression) provide quick wins, with LSTM as a stretch goal if time permits. All chosen libraries (scikit-learn, TensorFlow, Prophet, Transformers) are free and have abundant tutorials.

Backend Design

The backend is a **REST API server** built with FastAPI. Key design aspects:

- **API Endpoints:** Example endpoints include:
 - `GET /api/v1/heatmap?city={city}&date={YYYY-MM-DD}` – returns a JSON grid of LST or NDVI values for that day.
 - `GET /api/v1/aqi?city={city}&date={YYYY-MM-DD}` – returns sensor data (AQI/Pollutant levels) or interpolated grid.
 - `GET /api/v1/hotspots?city={city}&type=HCHO` – returns geo-coordinates of detected hotspots for the specified pollutant/metric.
 - `GET /api/v1/forecast?city={city}&type=temperature` – returns predicted values (e.g. a JSON time series for next 7 days).
 - `POST /api/v1/chat` – (phase 3) accepts a JSON message and returns the chatbot’s response.
- **Data Models:** Pydantic schemas define the request/response formats, ensuring type safety. For instance, a `HeatResponse` model might contain `date`, `coordinates`, and `lst_value`.
- **Database Access:** The API uses an ORM or direct SQLAlchemy/PostGIS queries to fetch data. Spatial queries (`ST_Intersects`, `ST_Within`) are used when filtering by location. For example, `/heatmap` may query all LST points within a city polygon. Indexing ensures queries are fast.
- **Business Logic:** The API implements any processing needed on-the-fly. For example, `/forecast` might load a pre-trained model and run inference on-the-fly, returning the results as JSON.
- **Caching:** To improve performance, results of expensive operations (like generating a full-city heatmap or ML predictions) can be cached in memory or a fast store (e.g. Redis) keyed by inputs. FastAPI’s built-in `@lru_cache` can be used for simple cases.

- **Security:** Use HTTPS (via TLS) for encryption. API keys (for OpenAQ, OpenWeather) are kept secret on the server. If user auth is added, OAuth or API tokens would be implemented. For now, we assume public read access.
- **Background Tasks:** FastAPI's `BackgroundTasks` or an external scheduler (cron) can update the data daily. For example, a background job could run at midnight to fetch the latest satellite imagery or sensor data and refresh database tables.
- **Error Handling & Logging:** The API will return clear error messages (e.g. "data not available for this date"). Logging is set up to track usage and errors for debugging and performance monitoring.

In summary, the backend orchestrates the data flow between the geospatial database, external data sources, and the frontend. FastAPI provides a clean, scalable way to define endpoints with automatic documentation (OpenAPI). For example, FastAPI advertises itself as *"a modern, fast (high-performance) web framework for building APIs"* [42†L311-L319], making it a fitting choice for serving our ML and map services.

Frontend Design

The frontend is a **React** single-page application (SPA) providing an intuitive UI:

- **Layout:** A full-width map pane (Leaflet or Mapbox) displays the spatial layers. A collapsible sidebar holds controls: dropdown to select *city*, date picker or slider for *date range*, and checkboxes to toggle layers (NDVI, LST, AQI, HCHO). Beneath or next to the map, a panel shows charts (e.g. line chart of temperature vs. time) and hotspots list.
- **Interactive Map:** Using `react-leaflet`, we load base tiles (OpenStreetMap). Overlay layers include:
 - **Heatmap Layer:** Display LST or AQI as a color-graded heatmap. Can use canvas or tile with grayscale data colored by a palette.
 - **Choropleth/Contours:** Optionally, display city ward polygons colored by average data (e.g. ward-wise average LST).
 - **Marker Layer:** If needed, show marker points for pollutant sensors or user-selected locations.
- **Legends:** A legend describing the heatmap color scale is shown on the map.
- **Controls & Filters:** Standard UI elements (React Bootstrap or Material-UI) are used. For example, a dropdown to select *city*, date range sliders, and toggle switches for data layers. These trigger new API calls to FastAPI when changed.
- **Charts and Panels:** We embed chart components (e.g. Chart.js, Recharts) to show time-series of data. For instance, after picking a city, the app fetches `/api/v1/timeseries?city=Nagpur&type=aqi` and plots it. On map-click, a popup or side panel could show historical trends at that point.

- **Responsiveness:** The UI is made responsive so it works on tablets/mobiles (using CSS flex/grid or a framework like Bootstrap).
- **State Management:** For simplicity, React's `useState` and `useEffect` hooks manage state (selected city, date). If it grows complex, Redux or Context API could be introduced, but likely not needed at MVP scale.
- **Error and Loading States:** The app shows spinners when data is loading and error messages if API calls fail (e.g. "No data for this date").

Overall, the frontend focuses on clarity and performance. We avoid heavy 3D graphics to keep it simple. As React is "declarative" and "component-based" [44†L16-L24], we build reusable components for the map, chart, and UI controls. The frontend simply calls the backend API and visualizes the results; all heavy computation remains server-side.

Database Design

Using PostgreSQL with PostGIS, the spatial database schema might include:

- **cities** (`city_id` PK, `name`, `state`, `country`, `geom` POLYGON): Stores the boundary (geometry) of each target city. Useful for filtering data to a city and computing aggregates.
- **air_quality_readings** (`aq_id` PK, `city_id` FK, `datetime`, `pollutant_type`, `value`, `geom` POINT): Raw sensor data from OpenAQ (or other). Each record includes the pollutant (e.g. PM2.5, O₃), its measured value, a timestamp, and location as a Point. Spatial index on `geom` enables quick lookup of readings within a region.
- **satellite_indices** (`sat_id` PK, `city_id` FK, `date`, `ndvi_avg` FLOAT, `lst_avg` FLOAT, `lst_max` FLOAT, ...): Daily aggregated values. For each city and date, store average NDVI, average/max LST, etc. This summary table speeds up queries for city-level trends.
- **heatmap_data** (`heat_id` PK, `city_id` FK, `datetime`, `geom` POINT, `ndvi` FLOAT, `lst` FLOAT): Optionally, precomputed gridded data points for map heatmaps. Each row might represent the center of a grid cell with its NDVI/LST values.
- **predictions** (`pred_id` PK, `city_id` FK, `date`, `type`, `pred_value`): Model forecasts. E.g. `type`='heat', `pred_value` is predicted max LST.
- **users** (if adding login) (`user_id`, credentials, preferences, etc.) – likely out of scope for MVP.
- **recommendations** (`rec_id` PK, `city_id` FK, `date`, `recommendation` TEXT): To store any auto-generated advice or alert messages.

Key points: The `geom` columns use PostGIS geometry types (POINT, POLYGON). Spatial indexes (GIST) are defined on these columns for fast geospatial queries [46†L17-L21] (e.g. finding all points within a city

polygon). Foreign keys link `air_quality_readings` and `predictions` to `cities` for relational integrity.

For example, to generate a city's daily average temperature, one could run a query joining `heatmap_data` filtered by city's `geom` and date range. PostGIS functions like `ST_Contains(cities.geom, heatmap_data.geom)` help with this. The database enables efficient filtering (e.g. get last month's LST data for city X) and supports the backend queries for API endpoints.

A concrete *suggested schema* (in pseudo-SQL) might be:

```
CREATE TABLE cities (  
  city_id SERIAL PRIMARY KEY,  
  name VARCHAR,  
  state VARCHAR,  
  geom GEOMETRY(POLYGON, 4326)  
);  
CREATE TABLE air_quality_readings (  
  aq_id SERIAL PRIMARY KEY,  
  city_id INT REFERENCES cities(city_id),  
  datetime TIMESTAMPTZ,  
  pollutant VARCHAR,  
  value FLOAT,  
  geom GEOMETRY(POINT, 4326)  
);  
CREATE TABLE satellite_indices (  
  sat_id SERIAL PRIMARY KEY,  
  city_id INT REFERENCES cities(city_id),  
  date DATE,  
  ndvi_avg FLOAT,  
  lst_avg FLOAT,  
  lst_max FLOAT  
);  
CREATE INDEX ON air_quality_readings USING GIST (geom);  
CREATE INDEX ON cities USING GIST (geom);
```

This structure can be extended as needed (e.g. separate tables for each pollutant, or join tables for many-to-many relationships). PostGIS's vector support (and even raster support) means the database can evolve; for MVP, storing aggregated indices and point readings is sufficient.

GIS and Geospatial Workflow

EarthLens AI's geospatial workflow leverages PostGIS and GIS libraries:

- **Projection Handling:** All spatial data use a common coordinate reference system (e.g. WGS84 EPSG: 4326 for simplicity). If Earth Engine returns data in another CRS, we apply `ST_Transform` to ensure consistency [\[46†L17-L21\]](#) .
- **Spatial Queries:** Use PostGIS functions in SQL. For example, to get all sensor readings in a city:

```
SELECT * FROM air_quality_readings AS aq
JOIN cities ON ST_Contains(cities.geom, aq.geom)
WHERE cities.name = 'Nagpur';
```

This finds points where the city polygon contains the sensor point. PostGIS's spatial index makes this fast.

- **Map Tile Generation:** For the frontend map, heatmaps or choropleth data are served by the API. The backend might generate GeoJSON or raster tiles. If needed, tools like `mbutil` or `gdal2tiles` could pre-generate map tiles for static layers (NDVI, LST) to speed up rendering.
- **Geoprocessing Tools:** For preprocessing outside GEE, use GDAL (osgeo) or Rasterio/GeoPandas. For example, if manual LST retrieval is done, GDAL can mosaic and reproject images. However, Earth Engine largely covers this. For vector data, GeoPandas can read shapefiles to import city boundaries.
- **Visualization:** The React frontend will use map libraries as mentioned. These can display GeoJSON from PostGIS via the API or load raster images via Web Mercator tiles. Styling (color scales for heatmaps) is handled client-side.
- **Spatial Analysis:** Advanced GIS analysis (e.g. computing Zonal Statistics of LST by ward) can be done by PostGIS or GDAL. For a solo developer, heavy GIS work is minimized by using Earth Engine for raster analysis and PostGIS for basic vector ops.

Overall, the workflow is: Data ingestion (via GEE/API) → Geoprocess/clean (masking, aggregating) → Store spatially-enabled tables → Serve to frontend or analytics via PostGIS queries. The use of PostGIS is justified as *"the de facto open-source geospatial database"* [\[46†L17-L21\]](#) , making EarthLens AI's spatial computations reliable and standard-compliant.

Deployment Plan

To keep costs low, deployment will use free or minimal-cost services:

- **Containerization:** The backend (FastAPI server and any background workers) will be Dockerized. A Docker image simplifies moving between environments (development laptop → cloud VM). We also containerize the React app (or serve it as static files).

- **Hosting:** Options include a small cloud VM or platform-as-a-service. For example, using a free-tier VM from AWS/GCP/Oracle Cloud or a starter instance on Heroku/DigitalOcean for the backend + database. PostgreSQL/PostGIS can run on the same VM (small cities data will fit in <1GB easily). The React static site can be hosted on GitHub Pages or Netlify at no cost.
- **Domain and SSL:** If a custom domain is desired, use a cheap domain registrar and free SSL (Let's Encrypt). Otherwise, default domain from the host is fine for a portfolio.
- **CI/CD:** Simple: commit changes to GitHub triggers GitHub Actions to rebuild and redeploy (e.g. pushing Docker image to a registry, pulling on the server, restarting the container). Given this is a single-dev project, even manual deploys (scp files, docker-compose up) are acceptable to begin with.
- **Monitoring & Logging:** Use simple logging to files or stdout. For uptime monitoring, a free service like UptimeRobot can ping the API. Given the scope, heavy monitoring isn't needed, but logs will help diagnose issues.
- **Cost Control:** All components (React, FastAPI, PostgreSQL, Earth Engine) can run on a single small server. Monthly costs can be near \$0 (using free tiers or minimal resources). Earth Engine itself runs externally and only requires internet access (no billable CPU hours for small usage). For heavier traffic (not expected), the architecture can later move to scalable cloud services.
- **Environment:** Use environment variables for API keys (OpenAQ, OpenWeather) and secrets. Store a schema (SQL) script for setting up PostGIS on a new server.

In summary, deployment is straightforward: one VPS/container runs the full stack, the React front-end is served statically (or from the same server), and everything is public-facing via HTTPS. This ensures the project remains a low-maintenance proof-of-concept.

MLOps or Model Update Flow

Even as a solo developer, establishing a basic MLOps flow is wise:

1. **Version Control for Data and Models:** Keep training data and model code in git (or use git-lfs for large data). Tag or date-stamp model versions.
2. **Scheduled Retraining:** If we implement forecasting models, schedule periodic retraining (e.g. monthly). This can be a cron job that pulls new data from the DB and retrains the LSTM. The new model weights are saved (e.g. to a `models/` directory or cloud storage) and then loaded by the API.
3. **Evaluation Tracking:** After each retraining, evaluate metrics (MAE, RMSE) on a validation set to ensure improvements (or detect data drift). Log these metrics (even into a simple CSV or a logging table in the DB). For example, if RMSE for temperature forecasts suddenly jumps, investigate data issues.
4. **Containerize Models:** Include model files in the Docker image or mount them from an external volume. When deploying a new model, rebuild the container or swap the file.

5. **Feature Flags:** If adding a major new model, use a feature flag (or separate endpoint) so the old model can be quickly restored if issues arise.
6. **Scalable Compute:** For initial MVP, training can run on the same server (assuming small data). Later, if models get heavy, one could use a cloud GPU instance temporarily. But to keep costs near zero, we will restrict model complexity (small LSTM) and accept longer training time on CPU.
7. **Data Backup:** Regularly back up the database (the PostGIS database). This could be as simple as a nightly `pg_dump` to a storage bucket.

The key is simplicity: a script or notebook automates the training/inference steps, and we keep a log of model versions. This ensures EarthLens AI's predictive components remain up-to-date as new satellite and sensor data arrive.

Security, Scalability, and Limitations

- **Security:** Since data is mostly public, security risks are modest. We should still secure the API: use HTTPS, validate inputs to prevent SQL injection, and hide API keys in the server environment. If user logins are added, passwords should be hashed (bcrypt) and JWT tokens used. We should enforce reasonable rate limits to prevent abuse of our free-tier APIs. No sensitive personal data is handled.
- **Scalability:** This is initially a single-user demo, so heavy scalability isn't needed. However, if usage grows (e.g. multiple cities or many concurrent users), we can scale by:
 - *Database:* Move to a managed PostgreSQL service or a larger instance.
 - *Load Balancing:* Deploy additional API instances behind a load balancer (e.g. on Kubernetes or multiple Docker containers).
 - *Caching:* Implement Redis caching of frequent queries (heatmaps for today's date, static city shapefiles).
 - *Earth Engine Quotas:* Google Earth Engine has usage limits for individual users. If more data is needed, we could batch queries or throttle calls.

All chosen tech has growth paths: React app and FastAPI server can be containerized in a cluster. PostGIS is proven to handle large spatial data. However, to stay within scope, we assume moderate usage.

- **Limitations:**
 - *Spatial Resolution:* Sentinel-5P's HCHO data is ~5.5 km resolution, so it cannot pinpoint very small sources. NDVI/LST (10–30m) can be masked by clouds or urban materials.
 - *Temporal Gaps:* Satellite revisits are days apart; heavy cloud cover means no recent data. Ground sensors (OpenAQ) might have missing readings. Forecasts are probabilistic and degrade with longer horizons.
 - *Computational Load:* Calculating high-res maps on-the-fly can be slow, so precomputation or caching is used. Real-time data streaming isn't feasible.
 - *Funding and Maintenance:* The project relies on free tiers; if usage grows, costs (database storage, API limits) may rise. As a one-dev project, addressing bugs or adding features will take time.

We acknowledge these limitations as part of the risk management, focusing on clear documentation and manageable scope so that the project remains sustainable for one developer.

Evaluation Metrics

We will evaluate EarthLens AI's predictive models and overall effectiveness using appropriate metrics:

- **Heat Model (Regression):**

- *RMSE (Root Mean Squared Error)* in °C on predicted maximum LST or heat index (lower is better).
- *MAE (Mean Absolute Error)* in °C.
- *R² (coefficient of determination)* to gauge how much variance is explained.

For example, the reference study achieved an R^2 of 0.98 for Delhi using LSTM [31†L68-L72]. We will benchmark against simpler baselines (e.g. "predict tomorrow's temp = today's temp").

- **AQI Model:**

- *MAE* in AQI points or $\mu\text{g}/\text{m}^3$ (depending on pollutant), since high accuracy in pollutant levels is critical for public health advisories.
- Possibly *MSLE* if we care about relative error (but likely not needed).

- **Hotspot Detection:**

- Treat hotspot identification as a classification problem (hotspot vs non-hotspot grid cell). We can use *precision/recall* or *F1-score* if we have labeled examples (e.g. known industrial areas).
- If not, we may use qualitative validation (e.g. does detected HCHO hotspot match factory location?).

- **Map Accuracy:**

- If ground truth is available (like official heat maps or sensor grid), we could compute pixel-level accuracy or intersection-over-union with generated heatmaps. However, this is hard for environmental data.

- **Performance Metrics:**

- *API latency*: Time to serve key endpoints (e.g. heatmap generation), ensuring usability. Aim for <1s for typical queries.
- *Uptime*: For a demo, track that the service is available (through logging or external monitors).

- **User Engagement (Future):**

- Once deployed, usage metrics (number of visits, map queries) can indicate interest.
- For the AI assistant, user satisfaction (even via informal survey) could be noted.

All ML evaluations will be documented (with train/test splits clearly defined, e.g. train on 2020-2024 data, test on 2025) to show model validity. Where applicable, we'll cite [31†L68-L72] again to align our metric choices with literature. For simplicity, we will initially report RMSE and MAE for both heat and AQI models.

Risks and Mitigation

- **Data Limitations:**

- *Risk:* Frequent cloud cover (monsoon) could leave us without recent NDVI/LST. *Mitigation:* Use compositing (e.g. median over several days) and fuse multiple data sources (combine Landsat and Sentinel data).
- *Risk:* Sparse ground sensors in smaller cities. *Mitigation:* Use satellite proxies (e.g. combine Sentinel-5P NO₂ as a proxy for urban emissions). Also allow missing data without breaking the app (display “no data” gracefully).

- **One-Person Bottlenecks:**

- *Risk:* The project's scope could overwhelm a solo dev. *Mitigation:* Rigidly stick to the roadmap. Use existing libraries (don't reinvent the wheel). Reserve complex features (AI assistant, multi-city rollout) for Phase 3+.
- *Risk:* Knowledge gaps (deep learning, GIS). *Mitigation:* Focus on learning essentials gradually. Use online resources and possibly simplify models (e.g. use regression if deep learning is too time-consuming).

- **API/Service Availability:**

- *Risk:* Reliance on third-party APIs (OpenAQ, OpenWeatherMap, Earth Engine) means possible downtime or rate limits. *Mitigation:* Cache data, design for fallbacks. E.g. if OpenWeather fails, use last available data. Keep requests under free-tier limits by batching.

- **Performance:**

- *Risk:* Generating high-resolution maps on demand could be slow. *Mitigation:* Pre-calc common time periods. Use vector data for coarse zoom-out (to reduce pixels). Profile slow queries and optimize (add DB indexes, use spatial indexing).

- **Security:**

- *Risk:* Web exposure could invite misuse (e.g. someone scraping our API). *Mitigation:* Implement a simple rate limiter (e.g. 100 requests/min per IP). Avoid exposing raw database.

- **User Expectations:**

- *Risk:* Stakeholders might expect near-real-time or extremely accurate forecasts. *Mitigation:* Clearly communicate limitations (satellite latency, model uncertainty). Design the UI to show confidence intervals if available.

Documenting these risks and their mitigations (for example, in the README or documentation) is also part of the project's value, showing professional planning.

Implementation Roadmap for a Single Developer

Phase 1 – Foundation (Months 1–3):

1. **Data Access Setup:** Register for Google Earth Engine, OpenAQ, OpenWeatherMap. Write simple scripts to pull NDVI, LST, AQI for one city. Verify data (e.g. plot NDVI on a map offline to sanity-check). [23†L68-L72] [25†L61-L65]
2. **Database Initialization:** Install PostgreSQL/PostGIS. Load city boundary (Nagpur) and initial data. Create schema (tables for readings and indices).
3. **Basic Backend:** Scaffold a FastAPI app. Implement endpoints for `GET /api/heatmap` and `/api/aqi` that query the database and return JSON of values or GeoJSON.
4. **Basic Frontend:** Set up a React app with a map. Call the `/heatmap` API to display an LST heatmap overlay. Ensure layers can be toggled.
5. **Initial UI Components:** Add a sidebar to select city (pre-populated with Nagpur), and display a basic time-series chart for the last month of temperature (fetched from an endpoint).
6. **Testing & Iteration:** Manually test the data pipeline and map. Fix bugs (e.g. alignment issues, missing data). Prepare documentation of data sources and code.

Phase 2 – Core Features (Months 4–6):

7. **Additional Layers:** Integrate NDVI map layer and real-time AQI overlay. For AQI, fetch the latest OpenAQ readings each day. Display pollutants (PM2.5, etc.) using colored circles.
8. **Spatial Analysis:** Implement hotspot detection queries (e.g. areas above percentile threshold). Show hotspots as highlights on the map.
9. **Prediction Models:** Develop a simple forecasting model (e.g. linear regression or Prophet) for next-week temperature. Expose it via `/forecast`. Display the forecast chart in the UI. (No need for LSTM yet unless time allows.)
10. **Recommendations Module:** Hard-code basic suggestions (e.g. “plant trees in X” where NDVI is low) as a first pass.
11. **Polishing:** Improve UX (loading spinners, legends, mobile layout). Add error handling in UI. Optimize database indexes for speed.
12. **Documentation and Deployment:** Write a README. Containerize and deploy the MVP on a free-tier host. Verify everything works in the cloud environment.

Phase 3 – Advanced Features (Months 7–12+):

13. **Advanced ML:** Replace baseline models with LSTM networks for temperature and AQI using TensorFlow. Retrain on historical data. Compare metrics (cite improvements).
14. **AI Assistant:** Integrate a chatbot frontend (e.g. a simple chat box). For example, use OpenAI's GPT API to answer questions by querying our data endpoints internally.
15. **More Cities:** Add 2–3 more cities. Refine data ingestion to loop over a list of cities. Tweak UI to let users switch cities.
16. **Optimization & Scalability:** Profile performance (backend response times). Introduce caching (Redis) if needed. Possibly migrate DB to a dedicated service for larger data.
17. **Additional Layers:** Add new pollutant layers (NO₂ from Sentinel-5P, if desired) and socio-economic data layers (if available).

18. **Final Deployment:** Polish the production system, ensure HTTPS and backups, and optionally publish a project website or open-source repository.

By following this roadmap, the solo developer builds EarthLens AI incrementally: first focusing on one city and simple features, then gradually adding complexity. Each phase should conclude with a working demo, allowing feedback and ensuring progress is tangible.

Expected Outcomes

Upon completion of the MVP, EarthLens AI will have:

- A functioning **geospatial dashboard** showing current/ historical urban heat and air quality for at least one city. Users can visualize NDVI and LST maps, see pollution readings, and view forecasts.
- A documented **data pipeline** (code and schema) that others can replicate or extend.
- Basic **machine learning models** with evaluated performance (benchmarked via RMSE, R^2).
- A self-contained **code repository/portfolio project** demonstrating full-stack skills: satellite data processing, ML modeling, API development, and interactive visualization.
- **Potential Impact:** City planners using EarthLens AI could identify priority areas for cooling measures, researchers could access new merged datasets, and citizens gain a better understanding of their environment.

With further phases, we expect EarthLens AI to become a more powerful tool: capable of predicting dangerous heat and pollution episodes, issuing preventive warnings, and scaling to multiple cities. Even as an MVP, it fills an important niche in India's urban climate response toolkit.

Future Scope

Longer-term enhancements (beyond Phase 3) could include:

- **Digital Twin Simulation:** Integrate climate models or “what-if” scenarios (e.g. simulate effects of adding parks or reducing traffic).
- **IoT Sensor Integration:** Accept real-time feeds from local sensor networks (if deployed), for hyperlocal accuracy.
- **Advanced Analytics:** Use explainable AI (SHAP analysis) to show which factors (e.g. humidity, built-up area) drive forecasts, as in some studies [36†L166-L175] .
- **Collaboration Features:** Enable multiple stakeholders to annotate maps or share insights (e.g. a planner marking a new project area).
- **Monetization / Expansion:** Though the project aims to be open-source, a mature version could serve as the basis for a startup offering analytics-as-a-service to cities beyond India.
- **Educational Use:** Package the platform as an educational tool for university courses on GIS, climatology, or data science.

As more data (e.g. higher-res satellites, or crowdsourced weather) becomes available and computation cost drops, EarthLens AI can naturally incorporate these. The modular design ensures new sensors or models can be plugged in. We can also envision expanding to other countries with similar urban heat challenges.

Resume and Portfolio Value

For the solo developer, building EarthLens AI is a **major portfolio highlight**. It showcases:

- **Full-Stack Development:** Experience with databases (PostGIS), backend (FastAPI), and frontend (React) – all highly sought skills.
- **Geospatial Expertise:** Working with satellite imagery and geospatial analysis is a niche skill, valuable for environmental and urban-tech positions.
- **Machine Learning:** Implementing forecasts (especially LSTM/RNN models) and integrating an AI assistant demonstrates advanced ML competency.
- **Systems Thinking:** Documenting architecture and deployment shows ability to design end-to-end systems, not just coding tasks.
- **Problem-Solving:** Addressing a real-world issue (urban heat/pollution) indicates socially relevant project experience.

Such a project aligns with final-year capstones or research projects. It can also be presented as a startup prototype. Including this on a resume signals that the developer can handle complex, multi-disciplinary challenges single-handedly. Employers will see proficiency in up-to-date technologies (Python, FastAPI, React, GIS) and the soft skill of managing scope and timeline.

Learning Resources and References

Official Documentation and Tutorials:

- *Google Earth Engine Guide:* Comprehensive docs and tutorial code for computing NDVI/LST using satellite data (see Earth Engine Data Catalog [23†L68-L72] [25†L61-L65]).
- *OpenAQ API Docs:* Guides for accessing global air quality data [6†L112-L120] (Quickstart and API reference at docs.openaq.org).
- *FastAPI Documentation:* Official docs (fastapi.tiangolo.com) explaining how to build APIs efficiently [42†L311-L319] .
- *React Documentation:* Official React docs (react.dev) for building UIs; see the Meta React overview [44†L16-L24] .
- *PostGIS Manual:* Documentation for spatial SQL functions and best practices (PostGIS reference [46†L17-L21]).

Research and Background:

- *Urban Heat and ML: "Smart cities, hot cities"* (Tomar et al., 2025) – an open-access paper showing LSTM forecasting of urban heat in Delhi [31†L68-L72] [36†L147-L152] . Good for understanding modeling approaches.
- *GIS for Urban Heat:* Esri blog on Indian cities' UHI analysis [14†L121-L124] (for conceptual context on NDVI and LST significance).
- *OpenAQ and Satellite Validation:* OpenAQ blog posts on satellite vs ground AQ data (for insight into combining sources).

Tools and Libraries:

- *GDAL/OGR*: Geospatial data processing (official gdal.org).
- *PostgreSQL/PostGIS Tutorial*: Beginners' guides (postgis.net/workshops/).
- *FastAPI Tutorial*: Typer and official guides (tiangolo's blog).
- *Time-Series ML*: Prophet documentation, TensorFlow LSTM tutorials.

These resources provide in-depth technical guidance and context. Throughout development, refer to the cited docs and articles to ensure EarthLens AI uses each technology correctly and stays aligned with best practices and state-of-the-art methods.

Key Citations from Sources: The above plan references key official sources and research, including Sentinel/Landsat data descriptions [25†L61-L65] [23†L68-L72] , Sentinel-5P HCHO documentation [17†L68-L74] [21†L112-L118] , OpenAQ platform description [6†L112-L120] , the LSTM study [31†L68-L72] , and posts on geospatial tech [14†L121-L124] [46†L17-L21] . These grounding references confirm that EarthLens AI's data choices and modeling strategies are sound and up-to-date.
